

PROJECT: MISSION CONTROL

Orbiter-7

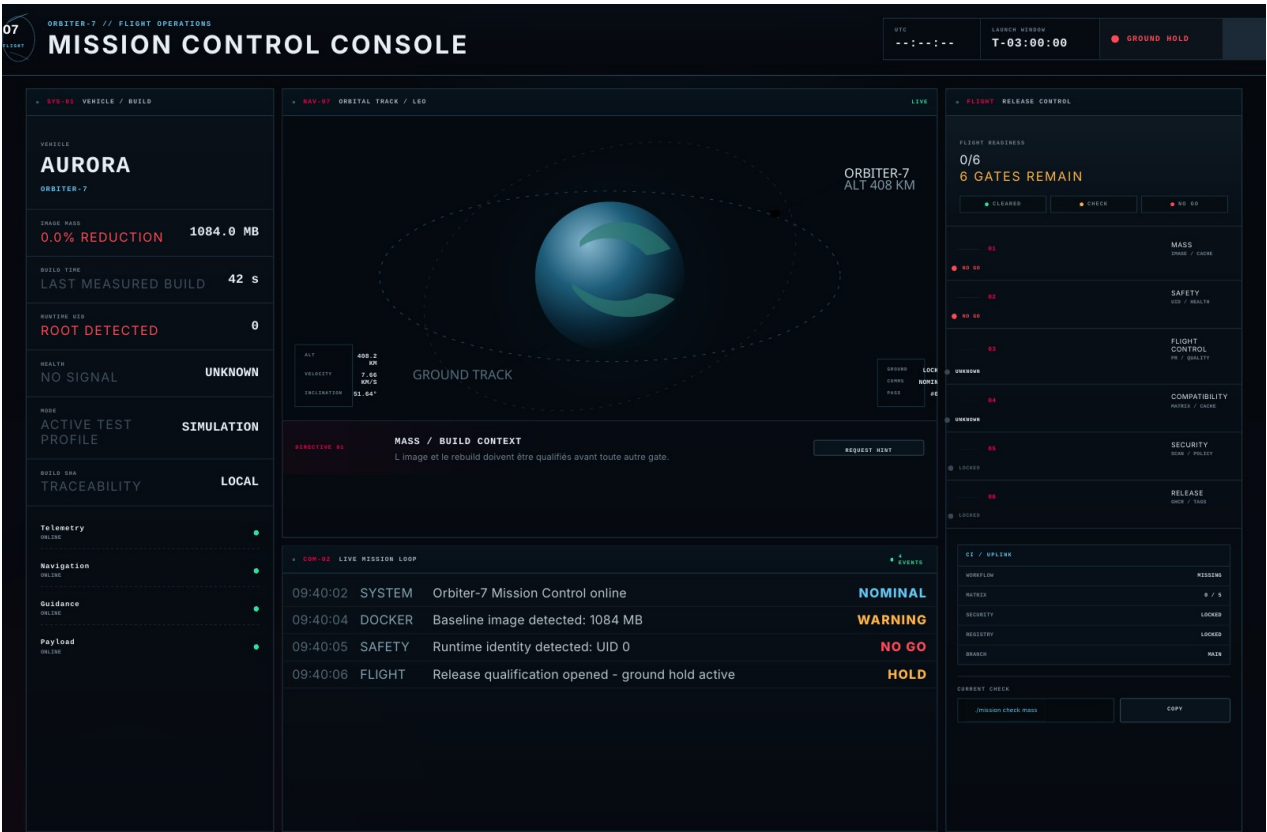
Qualifier une release sans recevoir la recette.

~3 heures parcours obligatoire	3 étudiants rotation des rôles	6 gates faits -> preuve -> GO
--	--	---

MISSION

Orbiter-7 fonctionne déjà. Dans trois heures, la fenêtre de lancement s'ouvre. Votre rôle n'est pas de refaire l'application : vous devez rendre sa chaîne de livraison suffisamment fiable, mesurable et sûre pour obtenir le GO.

Aperçu de Mission Control



Lancez Mission Control en 2 minutes

Les mêmes commandes fonctionnent sur macOS, Windows et Linux.

Terminal 1 - préparation

```
DEPUIS LE DOSSIER DU PROJET  
npm install  
npm run doctor  
npm run check  
npm start
```

CONSOLE

Quand le serveur indique qu'il écoute sur le port 3000, ouvrez <http://localhost:3000> et gardez Mission Control visible pendant tout le PLD.

Terminal 2 - premières commandes

```
MÊME DOSSIER  
npm run mission -- status  
npm run measure -- baseline
```

IMPORTANT

Vous n'avez besoin ni de chmod, ni de Bash, ni de WSL. Les outils du PLD passent par Node/npm pour rester identiques sur macOS, Windows et Linux.

Noms et emplacements à respecter

Les choix techniques sont à vous. Les chemins utilisés par les outils, eux, sont fixes.

Chemin exact	Action
TRAVAIL_ETUDIANT/Dockerfile	à modifier
TRAVAIL_ETUDIANT/Dockerfile.dockerignore	à créer - NOM EXACT
TRAVAIL_ETUDIANT/release-manifest.json	à modifier seulement pour le test du cache
TRAVAIL_ETUDIANT/FLIGHT_LOG.md	à compléter avec mesures et preuves
.github/workflows/mission.yml	à créer - NOM EXACT
TRAVAIL_ETUDIANT/PREUVES/	généré automatiquement - ne pas modifier à la main
TRAVAIL_ETUDIANT/release-gate.test.js	créé automatiquement par l'incident PR

DOCKER IGNORE

Ne créez pas un simple .dockerignore à la racine pour ce TP. Le fichier attendu est précisément TRAVAIL_ETUDIANT/Dockerfile.dockerignore.

ZONE PROTÉGÉE

SYSTEME_MISSION_NE_PAS_MODIFIER/ peut être lu pour comprendre le starter, mais ne doit pas être modifié. Votre travail principal reste dans TRAVAIL_ETUDIANT/ et .github/workflows/.

T-03:00 - la release est refusée

Le produit répond. Ce sont les conditions de livraison qui ne passent pas la revue.

RÈGLE DE MISSION

Pour chaque gate : observez le symptôme, formulez une hypothèse, faites le plus petit test utile, corrigez puis apportez une preuve. Le but n'est pas de deviner la consigne ni de recopier une recette.

Observer

Logs, taille, timings, identité, statut CI.
Commencez par un fait précis.

Décider

Choisissez une solution adaptée au problème. Plusieurs chemins peuvent être valides.

Prouver

Un check, une mesure ou un run GitHub doit confirmer que la gate est vraiment franchie.

NO GO

Ne désactivez jamais un test, un scan ou un contrôle pour obtenir du vert. Une pipeline verte obtenue en supprimant le garde-fou reste NO GO.

Une équipe de 3, trois postes de contrôle

Le clavier tourne, mais surtout la responsabilité de diagnostic.

Builder Exécute la modification choisie. Il ne tape pas avant que le groupe ait formulé l'hypothèse.	Flight Controller Lit les mesures et les logs. Il cherche le premier signal utile et propose le test suivant.	Safety Officer Challenge le raisonnement, tient le Flight Log et refuse toute gate sans preuve.
--	---	---

Rotation obligatoire

Phase	Builder	Controller	Safety
Gates 00-01	A	B	C
Gates 02-04	B	C	A
Gates 05-06	C	A	B

PEER LEARNING

Si vous connaissez déjà la réponse, votre travail n'est pas de la taper plus vite : amenez les deux autres à identifier le signal qui mène à cette réponse.

Bloqués ? Ouvrez un indice, pas la solution

Les indices sont progressifs : ils réduisent le champ de recherche sans faire le TP à votre place.

Hint 1 Observation : remet le groupe sur une piste sans nommer la technique.	Hint 2 Zone à inspecter : réduit fortement le champ de recherche.	Hint 3 Direction technique : le concept est presque donné, mais pas la correction finale.
--	---	---

RÈGLE

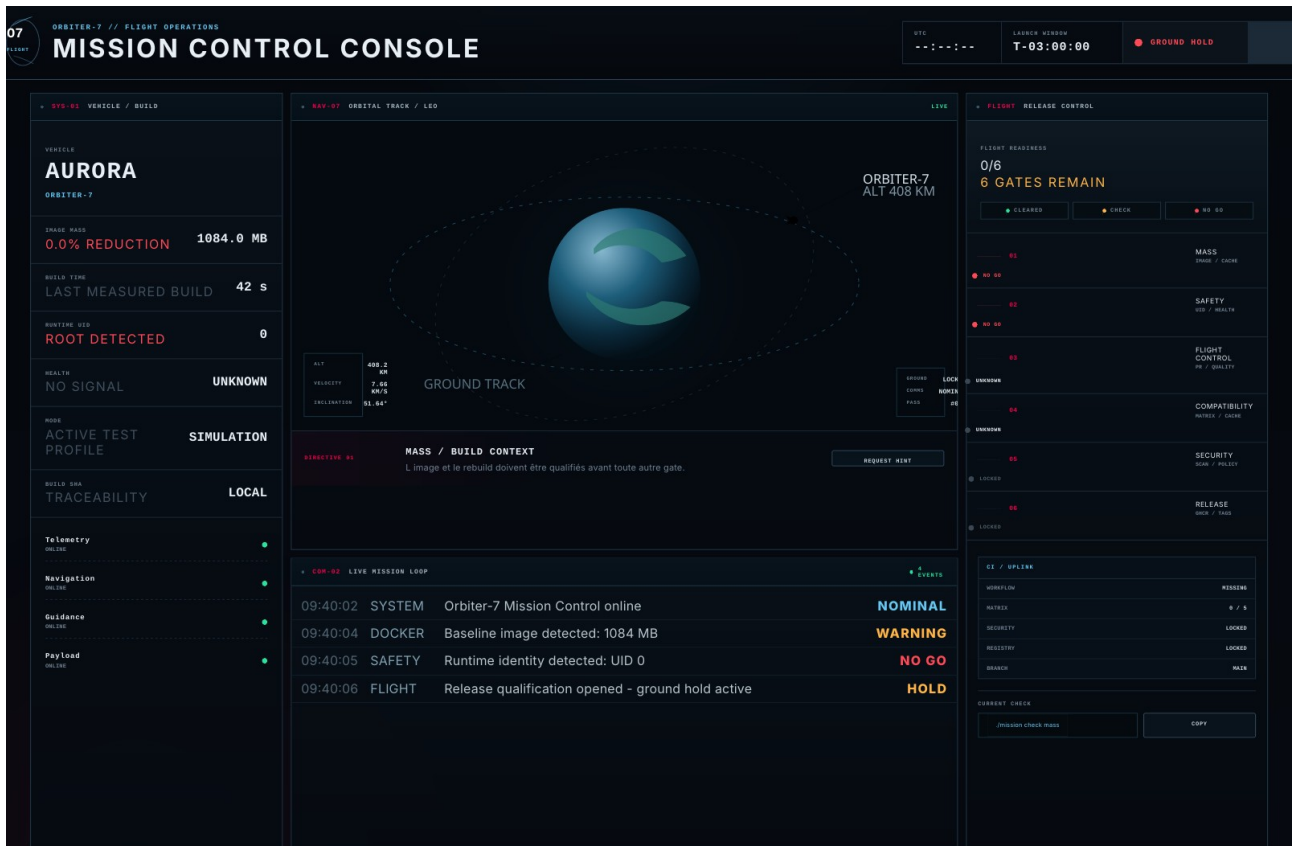
Commencez toujours par Hint 1. Discutez et faites un test. Hint 2 seulement si vous tournez encore en rond. Hint 3 est le dernier filet de sécurité.

Où les trouver

```
STARTER
SYSTEME_MISSION_NE_PAS_MODIFIER/indices/
gate-01-hint-1.md
gate-01-hint-2.md
gate-01-hint-3.md
...
```

La plateforme fait partie du TP

Gardez Mission Control ouverte : elle reflète vos mesures, vos checks, vos incidents et les indices demandés.



Boucle de travail

- Une mesure ou un check local alimente le LIVE MISSION LOOP.
- Cliquez une gate pour afficher la commande de contrôle correspondante.
- REQUEST HINT ouvre les indices progressifs et enregistre leur utilisation.
- Une anomalie injectée déclenche un MASTER ALARM ; une gate validée passe au vert.

IMPORTANT

Le dashboard n'invente pas du vert. Les gates locales viennent de checks réels ; les gates GitHub / GHCR demandent une preuve de run ou de package.

Ce que Mission Control exige

Les performances sont relatives à votre baseline. Les garanties de sécurité sont absolues.

Gate	Symptôme initial	GO
01 Mass	image lourde + rebuild lent	>= 60% de réduction + deps réutilisées
02 Safety	utilisateur root + health inconnu	non-root + healthy
03 Control	régression sans check PR	mauvaise PR bloquée
04 Compatibility	environnements non certifiés	5 combinaisons supportées vertes
05 Security	image publiable sans inspection	CRITICAL bloque la chaîne
06 Release	latest seul + mauvaise gate	publication tracée et contrôlée

À RETENIR

Le tableau vous dit ce qui doit être vrai à la fin. Il ne vous donne pas la recette pour y arriver.

180 minutes avant la fenêtre de lancement

Les Deep Space Incidents sont intégrés à Mission Control et ne se déverrouillent qu'après le GO obligatoire.

Temps	Mission
00:00 - 00:15	Gate 00 - baseline
00:15 - 00:50	Gate 01 - Mass
00:50 - 01:10	Gate 02 - Safety
01:10 - 01:45	Gate 03 - Flight Control
01:45 - 02:10	Gate 04 - Compatibility
02:10 - 02:35	Gate 05 - Security
02:35 - 02:55	Gate 06 - Release
02:55 - 03:00	GO / NO GO + Flight Log

CADENCE

Plus de 10 minutes sans nouveau fait = pause. Revenez au dernier état connu et ouvrez éventuellement le prochain indice.

Mesurez avant de toucher

Mission Control veut une photographie fiable de l'état initial.

CE QUI SE PASSE

Le produit fonctionne, mais personne ne sait encore combien pèse réellement l'image, combien de temps elle met à se construire ni avec quels privilèges elle tourne. Sans ces chiffres, vous ne pourrez pas prouver que vos futurs changements améliorent quoi que ce soit.

Ce que vous devez faire

- Vérifiez d'abord que le projet est sain avec les tests fournis et que Mission Control répond dans le navigateur.
- Construisez et mesurez la version initiale sans modifier le Dockerfile.
- Notez la taille de l'image, le temps de build et l'identité du process dans le container.
- Reportez ces valeurs dans TRAVAIL_ETUDIANT/FLIGHT_LOG.md : elles serviront de référence pour toutes les comparaisons suivantes.

Résultat attendu

- Une baseline chiffrée existe avant toute optimisation.
- L'application répond toujours sur `http://localhost:3000`.
- Le Flight Log contient les mesures initiales et l'utilisateur runtime.

Comment vérifier

```
npm run check
npm run measure -- baseline
npm run mission -- status
```

SI VOUS BLOQUEZ

À ce stade, ne cherchez pas encore à optimiser. Le but de cette gate est uniquement de connaître précisément le point de départ.

À RETENIR

Une amélioration sans baseline est une impression, pas une preuve.

Le vaisseau est trop lourd

L image fonctionne, mais sa construction coûte trop cher et transporte trop de choses.

CE QUI SE PASSE

L audit montre deux symptômes : l'image candidate est très lourde et un changement minuscule dans TRAVAIL_ETUDIANT/release-manifest.json provoque un rebuild presque aussi coûteux qu'un build complet. Le build context contient aussi des éléments qui n'ont aucun rôle au runtime.

Ce que vous devez faire

- Travaillez uniquement sur la façon dont l'image Docker est construite : ne modifiez pas l'application fournie.
- Analysez le Dockerfile actuel et l'ordre des étapes. Cherchez pourquoi une modification applicative oblige Docker à refaire autant de travail.
- Réduisez fortement ce qui est embarqué dans l'image finale tout en conservant uniquement ce qui est nécessaire au runtime.
- Créez le fichier TRAVAIL_ETUDIANT/Dockerfile.dockerignore avec ce nom exact et utilisez-le pour maîtriser ce qui part dans le build context.
- Après vos changements, vérifiez que l'interface et /health fonctionnent toujours.

Résultat attendu

- L'image finale est au moins 60% plus petite que la baseline.
- Un changement de code ou de manifest ne réinstalle pas inutilement les dépendances.
- Les fichiers sans utilité runtime ne se retrouvent pas dans l'image.
- Le produit continue de démarrer et de répondre normalement.

Comment vérifier

```
npm run measure -- final
npm run compare
# Modifiez uniquement une valeur dans TRAVAIL_ETUDIANT/release-manifest.json
npm run measure -- rebuild
npm run mission -- check mass
```

SI VOUS BLOQUEZ

Commencez par observer les étapes que Docker refait après la petite modification. Si vous ne voyez pas d'où vient l'invalidation, ouvrez Hint 1 puis Hint 2.

À RETENIR

Ne cherchez pas le record de la plus petite image. Le bon résultat est une image plus légère, reproductible et toujours exploitable.

Montrez ce qui a réellement changé

“C est plus rapide” ne suffit pas : Mission Control veut un avant/après compréhensible.

Dans votre Flight Log

- Taille baseline -> taille finale -> pourcentage de réduction.
- Temps du full build -> temps du code-only rebuild.
- Une trace montrant que la partie dépendances a été réutilisée.
- Une phrase simple du type : “nous avons changé X ; la mesure Y a évolué de Z”.

GO

La gate est valide si la réduction atteint au moins 60%, que les dépendances sont réellement réutilisées sur un changement applicatif et que l'application fonctionne toujours.

ATTENTION

Si vous réduisez la taille mais cassez la santé, les permissions ou le démarrage, ce n'est pas une optimisation acceptable.

Le process a trop de privilèges

L application répond, mais le runtime est encore trop permissif et Docker ne sait pas dire s il est réellement en bonne santé.

CE QUI SE PASSE

La baseline montre que le process tourne avec des privilèges élevés et que le container n expose pas de véritable état de santé à Docker. Dans un environnement de production, "le process existe" ne suffit pas pour considérer le service comme acceptable.

Ce que vous devez faire

- Inspectez l utilisateur qui exécute réellement l application dans le container.
- Adaptez la configuration Docker pour que l application tourne avec un utilisateur non-root sans casser les permissions nécessaires.
- Ajoutez un mécanisme de santé qui interroge réellement l endpoint /health et que Docker peut exécuter dans l image finale.
- Vérifiez que le mécanisme choisi repose sur un outil réellement présent dans l image que vous avez construite.
- Ne modifiez pas le code serveur dans SYSTEME_MISSION_NE_PAS_MODIFIER pour contourner le problème.

Résultat attendu

- Le runtime ne tourne plus avec UID 0.
- Docker finit par déclarer le container healthy.
- GET /health répond 200 et l interface reste accessible.

Comment vérifier

```
npm run mission -- check safety  
npm run mission -- status
```

SI VOUS BLOQUEZ

Commencez par inspecter l UID et le champ Health de Docker. Si le container démarre mais reste unhealthy, regardez précisément ce que le probe essaie d exécuter.

À RETENIR

Non-root et healthy sont deux garanties différentes : vous devez obtenir les deux.

Une régression arrive sans contrôle

Pour l'instant, une Pull Request peut être proposée sans qu'aucun contrôle automatique ne dise si elle casse le projet.

CE QUI SE PASSE

Un développeur ouvre une PR vers main. GitHub n'exécute aucun contrôle et la phrase "ça marche sur ma machine" est la seule garantie disponible. Votre mission est de créer un premier filet de sécurité automatique avant le merge.

Ce que vous devez faire

- Créez `.github/workflows/mission.yml` avec ce nom exact.
- Faites en sorte qu'une proposition de changement vers main déclenche automatiquement les contrôles utiles du projet.
- La chaîne doit au minimum vérifier la qualité du code et exécuter les tests existants sur une machine propre.
- Ne vous contentez pas d'avoir une pipeline verte : vous devrez ensuite provoquer une vraie régression pour vérifier qu'elle sait aussi refuser une mauvaise PR.

Résultat attendu

- Une PR saine obtient des contrôles verts.
- Une PR contenant la régression fournie obtient au moins un contrôle rouge.
- Après correction de la régression, la même protection revient au vert sans désactiver de test.

Comment vérifier

```
npm run incident -- pr
# Créez une branche, commitez, poussez puis ouvrez une Pull Request vers main.
```

SI VOUS BLOQUEZ

Les commandes npm déjà présentes dans `package.json` vous indiquent ce que le projet doit vérifier localement. Votre workflow doit reproduire ces garanties sur GitHub.

À RETENIR

Le challenge ici n'est pas d'écrire beaucoup de YAML : c'est de rendre une mauvaise modification visible avant main.

Prouvez que votre filet de sécurité existe

Une CI que vous n'avez jamais vue échouer peut être décorative.

Ce que l'incident fait

- La commande crée dans TRAVAIL_ETUDIANT/ un test volontairement incorrect.
- Votre branche contient donc une vraie régression testable, sans modifier le système protégé.
- Vous devez pousser cette branche et ouvrir une PR pour observer le comportement réel de GitHub Actions.

```
INJECTION
npm run incident -- pr
npm test

# Ensuite : branche courte -> commit -> push -> Pull Request vers main
```

Preuves obligatoires

- Lien ou capture du run GitHub rouge.
- Le log exact qui révèle la régression.
- Une correction du problème, pas une suppression du test ou du workflow.
- Un second run vert après correction.

BUT

Vous devez pouvoir expliquer ce qui empêchera désormais la même régression d'arriver silencieusement sur main.

Cinq environnements, une seule logique de test

La mission doit certifier exactement les environnements réellement supportés - pas toutes les combinaisons possibles.

CONTEXTE

Le même logiciel sera utilisé dans plusieurs environnements Mission Control. Ils n'utilisent pas tous la même version de Node ni le même mode de fonctionnement.

Environnement	Runtime	Mode
Ground Control	Node 20	simulation
Mission Simulator	Node 22	simulation
Flight Control A	Node 22	flight
Next-Gen Simulator	Node 24	simulation
Flight Control B	Node 24	flight

Ce que vous devez faire

- Faites évoluer votre CI pour exécuter la même logique de test sur ces cinq couples Node/mode.
- Évitez de copier-coller cinq jobs presque identiques : une seule définition doit piloter les différentes exécutions.
- Assurez-vous que la valeur du mode parvient réellement aux tests via MISSION_MODE.
- N'ajoutez pas Node 20 + flight : cette combinaison n'est pas annoncée comme supportée.

Résultat attendu

- Cinq exécutions distinctes apparaissent dans GitHub Actions.
- Les cinq sont vertes.
- Chaque exécution affiche clairement le runtime et le mode qu'elle certifie.

```
# Dans GitHub Actions, vous devez voir exactement :  
Node 20 / simulation  
Node 22 / simulation  
Node 22 / flight  
Node 24 / simulation  
Node 24 / flight
```

SI VOUS BLOQUEZ

Demandez-vous comment une seule définition de job peut recevoir plusieurs jeux de valeurs sans générer toutes les combinaisons possibles.

La CI ne doit pas tout recommencer à zéro

Vous avez maintenant plusieurs exécutions : si chacune réinstalle toujours toutes les dépendances, la pipeline devient inutilement lente.

CE QUI SE PASSE

La logique de test est la même d'un run à l'autre. Tant que le fichier qui décrit les dépendances ne change pas, GitHub doit pouvoir réutiliser le travail déjà fait au lieu de repartir complètement de zéro.

Ce que vous devez faire

- Ajoutez une stratégie de cache adaptée aux dépendances Node dans votre workflow.
- La réutilisation doit être sûre : un changement du lockfile doit pouvoir invalider le cache.
- Conservez une preuve d'un premier run qui remplit le cache puis d'un run ultérieur qui le réutilise.

Résultat attendu

- Un run ultérieur indique explicitement un cache hit ou une restauration de cache.
- Les tests continuent de tourner sur les cinq environnements.
- Un changement de dépendances ne doit pas conserver aveuglément un ancien état.

Comment vérifier

```
# Vérification principale : GitHub -> Actions -> logs du setup/install  
# Conservez le lien d'un run où le cache est réutilisé.
```

SI VOUS BLOQUEZ

Ne mesurez pas uniquement la durée totale : les runners et le réseau varient. Cherchez surtout la preuve que la donnée mise en cache a réellement été réutilisée.

Une image verte peut encore être dangereuse

Les tests et le build peuvent passer alors que l'image contient une vulnérabilité connue.

CE QUI SE PASSE

Votre pipeline sait maintenant vérifier le code et construire l'image, mais rien ne prend encore de décision de sécurité sur cette image avant sa publication. La politique de la mission est simple : une vulnérabilité CRITICAL connue doit pouvoir arrêter la release.

Ce que vous devez faire

- Ajoutez à la CI une étape qui inspecte l'image Docker réellement construite et recherche les vulnérabilités.
- Configurez la politique pour qu'une finding CRITICAL ne soit pas seulement affichée : elle doit rendre la chaîne rouge.
- Organisez les dépendances entre les jobs pour qu'une publication ne puisse démarrer que si le build puis le contrôle de sécurité ont réussi.
- Une publication doit également être impossible si le scan n'a pas été exécuté.

Résultat attendu

- Image acceptable -> le scan passe et la suite peut continuer.
- CRITICAL détectée -> le workflow échoue et la publication ne démarre pas.
- Scan absent ou sauté -> aucune publication production.

Comment vérifier

```
# Vérification principale : GitHub Actions
# Observez l'ordre réel : build -> contrôle sécurité -> publication
# Conservez au moins un run qui prouve que la sécurité peut stopper la chaîne.
```

SI VOUS BLOQUEZ

Si votre outil de scan affiche bien une CRITICAL mais que GitHub reste vert, votre contrôle est décoratif. Cherchez ce qui décide réellement du code de sortie et de la suite du workflow.

À RETENIR

La sécurité n'est pas un rapport à lire après publication : c'est une décision située avant publication.

Le scan doit réellement gouverner la suite

L ordre des jobs fait partie de la garantie de sécurité.

Questions à pouvoir répondre

- Si le build échoue, pourquoi le scan ne peut-il pas inspecter l image attendue ?
- Si le scan échoue, quel job doit forcément rester bloqué ?
- Qu est-ce qui prouve qu une CRITICAL est bloquante et pas seulement visible dans les logs ?
- Que se passe-t-il si le job de publication possède ses propres credentials mais aucune dépendance envers le scan ?

GO

Vous avez une exécution qui démontre que la sécurité a un vrai pouvoir d arrêt sur la release.

Publier, oui - mais uniquement au bon moment

Une PR doit pouvoir être validée sans pour autant écrire dans le registry de production.

CE QUI SE PASSE

Mission Control sait maintenant vérifier la qualité, la compatibilité et la sécurité. Il reste à publier l'image. Deux risques sont encore présents : publier depuis un événement qui n'est pas autorisé et ne conserver que latest, qui ne permet pas de retrouver précisément le commit exécuté.

Ce que vous devez faire

- Ajoutez l'authentification au registry sans écrire de secret ou de token en clair dans le workflow.
- Autorisez la publication uniquement depuis le chemin de release prévu (par exemple le push sur la branche principale et les tags de version si vous les supportez).
- Une Pull Request doit pouvoir construire et tester sans publier en production.
- Générez au minimum un tag latest pour la branche de release et un tag lié au SHA du commit.
- La publication doit rester située après la gate de sécurité.

Résultat attendu

- Une PR exécute les validations mais le job de publication est skipped.
- Un push autorisé sur main produit une image dans GHCR.
- Le package possède latest et un tag basé sur le SHA.
- À partir d'une image publiée, vous pouvez retrouver le commit exact qui l'a produite.

Comment vérifier

```
# Vérification : GitHub -> Actions puis GitHub -> Packages / GHCR  
# Conservez le lien du run de publication et le lien du package.
```

SI VOUS BLOQUEZ

Comparez les informations disponibles dans github.event_name, github.ref et github.sha : elles permettent de distinguer une PR, une branche et un commit précis.

À RETENIR

latest est pratique pour consommer la dernière release, mais ce n'est pas une preuve de traçabilité.

Montrez exactement ce qui a été publié

Une stratégie de tags est une interface de traçabilité.

RÉSULTAT ATTENDU

```
ghcr.io/<owner>/orbiter-7:latest  
ghcr.io/<owner>/orbiter-7:sha-<commit>
```

Bonus après GO

```
ghcr.io/<owner>/orbiter-7:v1.0.0
```

Dans le Flight Log

- Lien vers le package GHCR.
- Lien vers le run qui a publié.
- SHA du commit correspondant à une image.
- Preuve qu'une PR n'a pas exécuté la publication production.

GO

La chaîne est à la fois contrôlée et traçable : on sait quand elle a publié, pourquoi elle avait le droit de publier et quel commit se trouve dans l'image.

Le chemin vers le GO

Chaque flèche doit pouvoir être expliquée par les trois membres du groupe.



LECTURE

Qualité et compatibilité doivent être vertes avant le build de release. La sécurité peut bloquer. La publication est la dernière conséquence, jamais le premier réflexe.

Questions de groupe

- Quel est le premier endroit où une mauvaise PR peut être stoppée ?
- Quel est le dernier endroit où une image dangereuse peut encore être stoppée ?
- Quelle preuve permet de retrouver exactement ce qui a été publié ?

T-05 - dernier contrôle

Pas de Deep Space tant que cette liste n'est pas complète.

- ☐ Baseline mesurée avant optimisation
- ☐ Réduction image $\geq 60\%$
- ☐ Code-only rebuild réutilise les dépendances
- ☐ Runtime non-root
- ☐ Docker annonce healthy
- ☐ PR rouge réellement observée
- ☐ PR corrigée réellement verte
- ☐ 5 combinaisons de compatibilité vertes
- ☐ MISSION_MODE testé
- ☐ Cache de dépendances prouvé
- ☐ Image construite en CI
- ☐ CRITICAL peut bloquer
- ☐ Publish dépend de la sécurité
- ☐ PR ne publie pas production
- ☐ latest + SHA disponibles
- ☐ Flight Log contient les preuves

GO CONDITION

Les seize preuves sont présentes et chaque membre peut expliquer au moins une gate qu'il n'a pas lui-même implémentée.

Orbiter-7 - GO FOR LAUNCH

Le produit n a presque pas changé. La confiance autour de sa livraison, oui.



GOOD LAUNCH

Poussez les derniers commits, gardez le Flight Log lisible et laissez une chaîne que la prochaine équipe peut comprendre.

12 questions à pouvoir expliquer

Répondez avec un exemple de votre mission, pas avec une définition apprise par cœur.

1. Pourquoi mesurer avant d optimiser ?
2. Qu est-ce qui rend un code-only rebuild rapide ?
3. Pourquoi une image plus petite peut rester mauvaise ?
4. Que garantit réellement un runtime non-root ?
5. Process alive et healthy : quelle différence ?
6. Pourquoi provoquer volontairement une PR rouge ?
7. Pourquoi les cinq combinaisons ne sont-elles pas un simple tableau 3 x 2 ?
8. Comment savez-vous que le cache a servi ?
9. Pourquoi le scan doit pouvoir échouer ?
10. Pourquoi l ordre build -> scan -> publish compte ?
11. Pourquoi latest ne suffit pas ?
12. Quel changement a eu le plus d impact et comment le prouvez-vous ?

Vous avez du temps ? Maintenant ça se complique

Ces scénarios sont facultatifs. Après le GO FOR LAUNCH, la plateforme déverrouille automatiquement Deep Space Mode.

Dans la plateforme

Flight Readiness 6/6 -> DEEP SPACE MODE se déverrouille. X1 devient AVAILABLE ; X2 à X6 restent LOCKED.

Démarrez la mission disponible avec `npm run incident -- X1`. Elle passe ACTIVE et devient la directive courante.

Quand la correction et les preuves sont prêtes : `npm run mission -- check X1`. Si X1 passe CLEARED, X2 devient AVAILABLE automatiquement.

Même boucle jusqu'à X6. Les bonus sont post-lancement : ils ne retirent jamais le GO FOR LAUNCH.

Règle 01	Règle 02	Règle 03
Un incident à la fois. Gardez les gates obligatoires intactes.	Commencez par expliquer pourquoi le système actuel est dangereux ou trompeur.	Une correction sans preuve ne valide pas l'incident.

CHOIX

Le staff peut vous attribuer un incident ou vous laisser choisir selon votre avance. Les fichiers complets sont dans `SYSTEME_MISSION_NE_PAS_MODIFIER/incidents_bonus/`.

Le scanner est là... mais ne décide rien

Bonus avancé - commencez par comprendre le risque avant de modifier.

CE QUI SE PASSE

Un scan de vulnérabilités apparaît bien dans la pipeline. Pourtant une CRITICAL est visible dans le rapport, le workflow reste vert et l'image est quand même publiée.

Ce que vous devez faire

- Identifiez pourquoi le scan n'a aucun pouvoir de blocage malgré la finding CRITICAL.
- Corrigez uniquement le contrôle de sécurité et la dépendance de publication : ne masquez pas la vulnérabilité.
- Conservez le comportement normal quand le scan est réellement vert.

Résultat attendu

- Une finding bloquante rend le workflow rouge.
- Le job publish ne démarre pas tant que la sécurité est rouge.
- Vous pouvez expliquer quelle configuration transformait le scan en simple décoration.

Comment vérifier

Preuve : `npm run mission -- evidence X1 <blocked-run-url>` puis `npm run mission -- check X1`

Provoquez ou utilisez un run contenant la finding bloquante. Montrez dans GitHub Actions que publish est skipped / bloqué.

SI VOUS BLOQUEZ

Si vous bloquez, revenez au premier signal qui prouve le problème. Les bonus demandent surtout de distinguer un symptôme visible de la garantie réellement cassée.

Une Pull Request écrit dans le registry de production

Bonus avancé - commencez par comprendre le risque avant de modifier.

CE QUI SE PASSE

La pipeline de validation fonctionne, mais elle traite une Pull Request comme une vraie release : une contribution non mergée peut écrire dans GHCR.

Ce que vous devez faire

- Repérez quelle condition ou absence de condition autorise publish pendant une PR.
- Gardez les étapes de validation utiles sur PR : build et scan peuvent continuer à servir de preuve.
- Réservez la publication production aux événements réellement autorisés par votre politique de release.

Résultat attendu

- PR : validation possible, publication production impossible.
- Push/release autorisé : publication possible après toutes les gates.
- Les credentials ne sont utilisés que lorsque publish a réellement le droit de s'exécuter.

Comment vérifier

Preuve : `npm run mission -- evidence X2 <pr-run-url>` puis `npm run mission -- check X2`

Ouvrez une PR et montrez `publish skipped`. Puis montrez un run de release autorisé qui publie correctement.

SI VOUS BLOQUEZ

Si vous bloquez, revenez au premier signal qui prouve le problème. Les bonus demandent surtout de distinguer un symptôme visible de la garantie réellement cassée.

Le lockfile change, le vieux cache survit

Bonus avancé - commencez par comprendre le risque avant de modifier.

CE QUI SE PASSE

La CI est rapide, mais elle réutilise encore un ancien cache après modification des dépendances. Le pipeline peut donc tester un état différent de celui décrit par le repository.

Ce que vous devez faire

- Déterminez quelle information doit participer à l'identité du cache.
- Conservez le cache hit quand les dépendances sont réellement identiques.
- Faites en sorte qu'une modification du lockfile force une nouvelle installation cohérente.

Résultat attendu

- Lockfile identique -> cache réutilisable.
- Lockfile modifié -> ancien cache non considéré comme équivalent.
- Les tests s'exécutent ensuite avec les dépendances correspondant au nouveau lockfile.

Comment vérifier

Preuves : `npm run mission -- evidence X3 <cache-hit-url> <cache-miss-url>` puis `npm run mission -- check X3`

Comparez un run avec cache hit et un run après modification du lockfile. Les logs doivent montrer l'invalidation ou une nouvelle clé.

SI VOUS BLOQUEZ

Si vous bloquez, revenez au premier signal qui prouve le problème. Les bonus demandent surtout de distinguer un symptôme visible de la garantie réellement cassée.

La même release doit aussi tourner sur ARM64

Bonus avancé - commencez par comprendre le risque avant de modifier.

CE QUI SE PASSE

Le registry contient aujourd'hui une image pour une seule architecture. Une partie de l'infrastructure Mission Control utilise linux/arm64 alors que d'autres machines utilisent linux/amd64.

Ce que vous devez faire

- Faites produire la même version de l'image pour les deux architectures demandées.
- Publiez une référence unique que Docker peut résoudre vers l'image adaptée à la machine cliente.
- Ne remplacez pas la release existante par deux tags manuels sans lien entre eux.

Résultat attendu

- linux/amd64 est disponible.
- linux/arm64 est disponible.
- Une inspection du manifest publié montre les deux plateformes sous la même référence.

Comment vérifier

Preuve : `npm run mission -- evidence X4 <manifest-url>` puis `npm run mission -- check X4`

Inspectez l'image publiée avec un outil Docker capable d'afficher les plateformes du manifest et conservez la sortie.

SI VOUS BLOQUEZ

Si vous bloquez, revenez au premier signal qui prouve le problème. Les bonus demandent surtout de distinguer un symptôme visible de la garantie réellement cassée.

Le record de taille a cassé l'observabilité

Bonus avancé - commencez par comprendre le risque avant de modifier.

CE QUI SE PASSE

Une équipe a voulu produire l'image la plus petite possible. L'application démarre, mais le healthcheck n'est plus exécutable dans l'image finale et Docker finit par déclarer le container unhealthy.

Ce que vous devez faire

- Identifiez ce qui manque réellement au probe de santé dans l'image finale.
- Rétablissez un healthcheck fiable sans revenir automatiquement à une énorme image de base.
- Comparez la taille et la santé avant/après votre correction.

Résultat attendu

- Le container devient healthy.
- Le probe utilise un outil réellement disponible au runtime.
- L'image reste raisonnablement légère et vous pouvez expliquer le compromis choisi.

Comment vérifier

Pas d'URL : Mission Control lance l'image et attend healthy. Validez avec `npm run mission -- check X5`

Lancez le container, attendez l'état Health puis comparez `docker inspect` et la taille de l'image avant/après.

SI VOUS BLOQUEZ

Si vous bloquez, revenez au premier signal qui prouve le problème. Les bonus demandent surtout de distinguer un symptôme visible de la garantie réellement cassée.

Quatre environnements verts, un rouge

Bonus avancé - commencez par comprendre le risque avant de modifier.

CE QUI SE PASSE

La définition de CI est valide et quatre environnements passent. Un seul couple Node/mode échoue systématiquement : supprimer cette combinaison rendrait la pipeline verte, mais masquerait peut-être un vrai problème.

Ce que vous devez faire

- Isolez la combinaison qui échoue et l'erreur exacte.
- Reproduisez le problème avec le plus petit cas possible avant de toucher à la définition des environnements supportés.
- Décidez, preuves à l'appui, s'il s'agit d'un défaut de workflow ou d'une incompatibilité réelle du code/runtime.

Résultat attendu

- La cause est classée : CI/configuration ou application/runtime.
- La correction cible la cause réelle au lieu de supprimer arbitrairement l'environnement.
- Les cinq environnements attendus redeviennent cohérents avec la politique de support.

Comment vérifier

Preuves : `npm run mission -- evidence X6 <red-run-url> <green-run-url>` puis `npm run mission -- check X6`

Documentez la combinaison, le message d'erreur, votre hypothèse et la commande ou le test minimal qui reproduit la panne.

SI VOUS BLOQUEZ

Si vous bloquez, revenez au premier signal qui prouve le problème. Les bonus demandent surtout de distinguer un symptôme visible de la garantie réellement cassée.

Indices progressifs

Ne lisez que l'indice dont vous avez besoin.

GATE 01 / HINT 1

Changez uniquement la valeur note dans TRAVAIL_ETUDIANT/release-manifest.json et observez les étapes Docker qui sont reconstruites.

GATE 01 / HINT 2

Regardez ce qui se passe avant l'installation des dépendances et ce qui peut invalider cette couche.

GATE 01 / HINT 3

Séparez ce qui décrit les dépendances de ce qui change souvent. Inspectez aussi le build context et les fichiers réellement nécessaires au runtime.

GATE 02 / HINT 1

Inspectez l'UID dans le container puis le champ Health dans docker inspect.

GATE 02 / HINT 2

L'image Node contient déjà un utilisateur non-root ; votre probe n'a pas forcément besoin d'installer un nouvel outil.

GATE 02 / HINT 3

Un utilisateur runtime non-root et un HEALTHCHECK vers /health avec un outil garanti dans l'image peuvent satisfaire la gate.

Indices progressifs

Ne lisez que l'indice dont vous avez besoin.

GATE 03 / HINT 1

La régression doit être détectée avant main. Quel événement GitHub correspond à une proposition de changement ?

GATE 03 / HINT 2

Le projet sait déjà exécuter lint et tests localement. Votre CI doit retrouver ces signaux sur une machine propre.

GATE 03 / HINT 3

Le fichier attendu est .github/workflows/mission.yml. Un workflow sur pull_request peut checkout, installer, lint et tester.

GATE 04 / HINT 1

Vous avez cinq combinaisons supportées, pas toutes les combinaisons possibles.

GATE 04 / HINT 2

Une seule définition de job peut recevoir des valeurs différentes via une stratégie d'exécution.

GATE 04 / HINT 3

Une stratégie avec une liste explicite des couples peut décrire exactement les 5 environnements ; passez MISSION_MODE aux tests et activez le cache npm.

Indices progressifs

Ne lisez que l'indice dont vous avez besoin.

GATE 05 / HINT 1

Un build vert n'est pas une décision de sécurité. Quelle vérification doit avoir lieu avant publish ?

GATE 05 / HINT 2

La finding bloquante doit produire un échec du job et le job suivant doit dépendre de cette réussite.

GATE 05 / HINT 3

Construisez l'image en CI, scannez-la avec un outil de vulnérabilités, faites bloquer CRITICAL et placez publish en aval.

GATE 06 / HINT 1

latest répond "la plus récente", pas "quel commit exact ?". Comparez aussi PR et push main.

GATE 06 / HINT 2

Pensez événement, Git ref, SHA et dépendance au job sécurité.

GATE 06 / HINT 3

Publiez seulement depuis le chemin autorisé après scan et générez au minimum latest + un tag basé sur le SHA.